

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский национальный исследовательский политехнический
университет»

Электротехнический факультет

Кафедра: Информационные технологии и автоматизированные системы
(ИТАС)

Направление подготовки: 09.03.04 – «Программная инженерия»

ОТЧЕТ

Тема: «Простые методы поиска»

По дисциплине «Основы алгоритмизации и программирования»

Выполнила: студентка группы РИС-25-26

Лаптева Елена Владимировна

Принял: доц. Полякова О.А.

Пермь, 2026

I. Лине́йный поиск

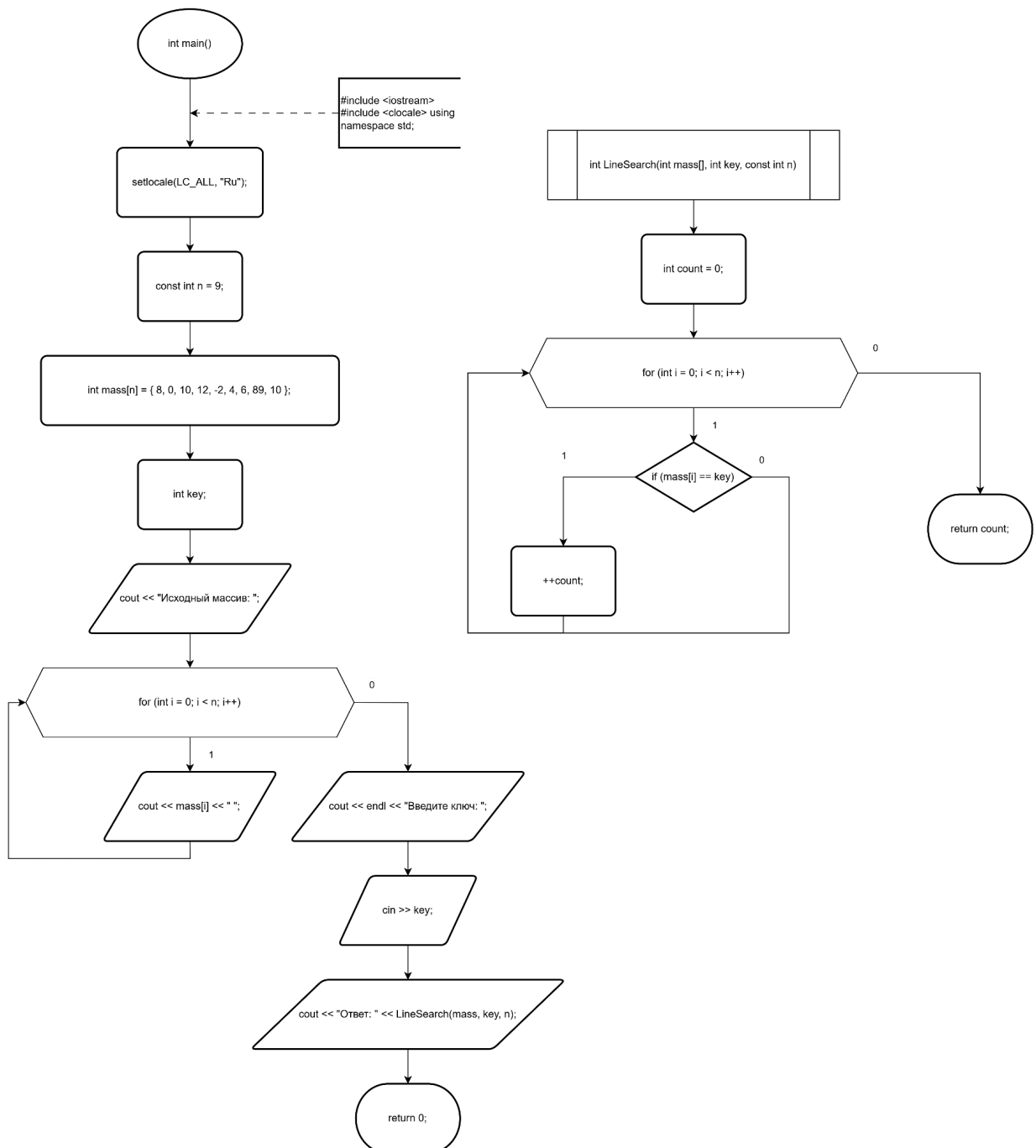
1. Постановка задачи

Задан массив целых чисел, пользователь должен ввести ключ. Необходимо найти сколько элементов с заданным ключом находятся в массиве.

2. Анализ задачи

1. Последовательный перебор каждого элемента массива в цикле
2. Сравнение каждого элемента с ключом поиска
3. Увеличиваем счетчик, если ключ совпал с элементом массива

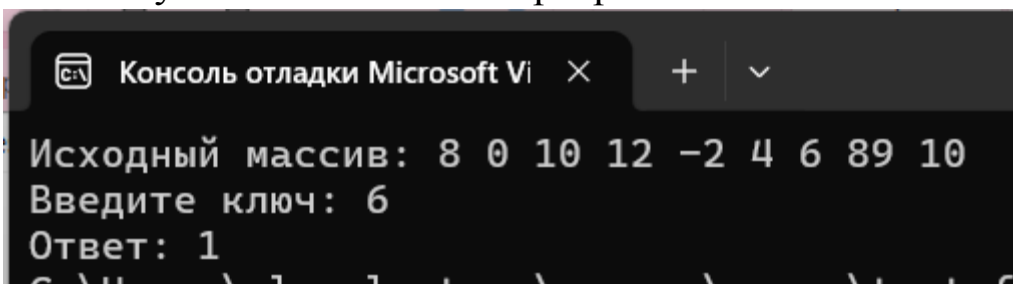
3. Блок-схема



4. Код

```
1. #include <iostream>
2. #include <locale>
3. using namespace std;
4.
5. int LineSearch(int mass[], int key, const int n)
6. {
7.     int count = 0;
8.     for (int i = 0; i < n; i++)
9.     {
10.         if (mass[i] == key)
11.         {
12.             ++count;
13.         }
14.     }
15.     return count;
16. }
17.
18. int main()
19. {
20.     setlocale(LC_ALL, "Ru");
21.     const int n = 9;
22.     int mass[n] = { 8, 0, 10, 12, -2, 4, 6, 89, 10 };
23.     int key;
24.
25.     cout << "Исходный массив: ";
26.     for (int i = 0; i < n; i++)
27.     {
28.         cout << mass[i] << " ";
29.     }
30.
31.     cout << endl << "Введите ключ: ";
32.     cin >> key;
33.     cout << "Ответ: " << LineSearch(mass, key, n);
34.
35.     return 0;
36. }
```

5. Результат выполнения программы



The screenshot shows the Microsoft Visual Studio debug console. The title bar reads "Консоль отладки Microsoft Vi". The output text is as follows:

```
Исходный массив: 8 0 10 12 -2 4 6 89 10
Введите ключ: 6
Ответ: 1
```

At the bottom, a partial file path is visible: "C:\Users\user\source\repos\test\test\src\main.cpp".

II. Бинарный поиск

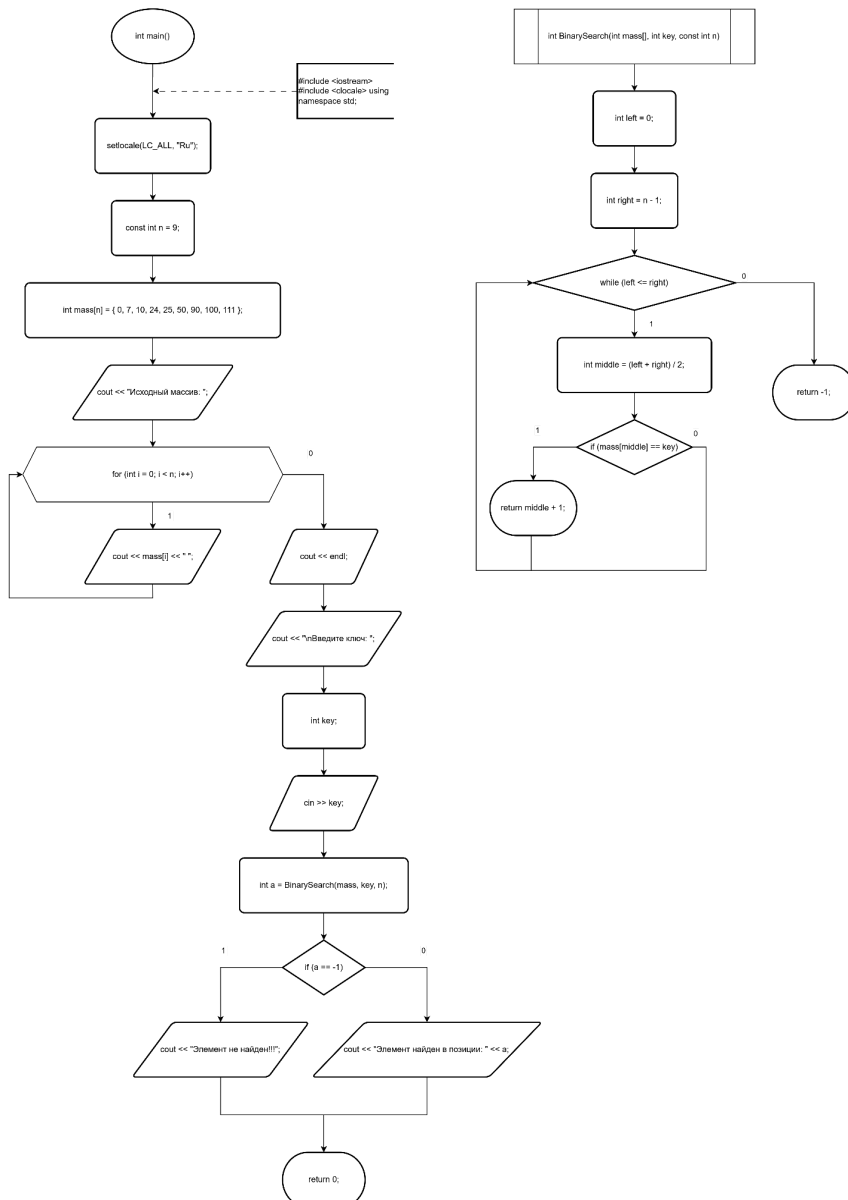
1. Постановка задачи

Найти номер элемента массива, значение которого равно ключу.

2. Анализ задачи

1. Находим границы множества
2. Находим середину множества
3. Сравниваем ключ с серединным элементом:
 - Если искомое значение элемента меньше серединного, то продолжается поиск в левой части множества
 - Если искомое значение элемента больше серединного, то продолжается поиск в правой части множества

3. Блок-схема

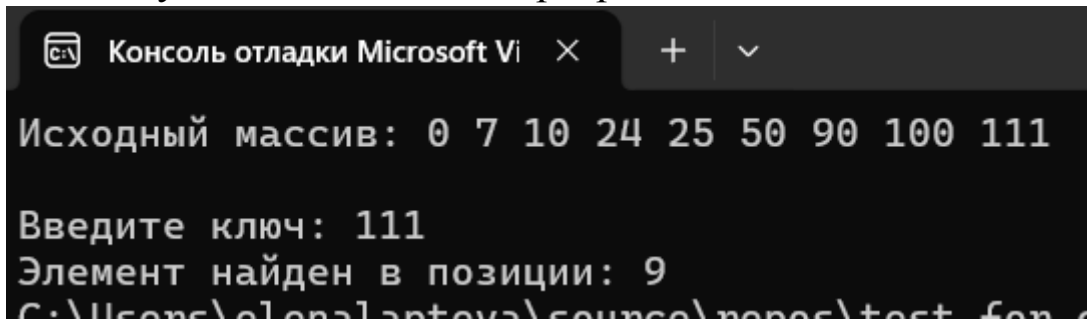


4. Код

```
1. #include <iostream>
2. #include <locale>
3. using namespace std;
4.
5. int BinarySearch(int mass[], int key, const int n) {
6.     int left = 0;
7.     int right = n - 1;
8.
9.     while (left <= right) {
10.         int middle = (left + right) / 2;
11.
12.         if (mass[middle] == key) {
13.             return middle + 1;
14.         }
15.         else if (mass[middle] < key) {
16.             left = middle + 1;
17.         }
18.         else {
19.             right = middle - 1;
20.         }
21.     }
22.     return -1;
23. }
24.
25. int main() {
26.     setlocale(LC_ALL, "Ru");
27.     const int n = 9;
28.     int mass[n] = { 0, 7, 10, 24, 25, 50, 90, 100, 111 };
29.
30.     cout << "Исходный массив: ";
31.     for (int i = 0; i < n; i++) {
32.         cout << mass[i] << " ";
33.     }
34.     cout << endl;
35.
36.     cout << "\nВведите ключ: ";
37.     int key;
38.     cin >> key;
39.
40.     int a = BinarySearch(mass, key, n);
41.
42.     if (a == -1) {
43.         cout << "Элемент не найден!!!";
44.     }
```

```
45.     else {  
46.         cout << "Элемент найден в позиции: " << a;  
47.     }  
48.  
49.     return 0;  
50. }  
51.
```

5. Результат выполнения программы



```
Консоль отладки Microsoft Vi × + ▾  
Исходный массив: 0 7 10 24 25 50 90 100 111  
Введите ключ: 111  
Элемент найден в позиции: 9  
C:\Users\elena\appdata\source\repos\test_for_c
```

III. Интерполяционный поиск

1. Постановка задачи

Найти номер элемента массива, значение которого равно ключу.

2. Анализ задачи

1. Определяется интерполяционный индекс (по формуле) для нахождения приблизительного положения искомого элемента в массиве.

1. Формула в интерполяционном поиске берётся из уравнения прямой, проведённой через две точки на числовой оси, и основана на принципе линейной интерполяции.

2. Изначальная функция:

$$\frac{x - y_0}{y_1 - y_0} = \frac{i - x_0}{x_1 - x_0}$$

где:

x — искомое значение (например, ключ поиска),

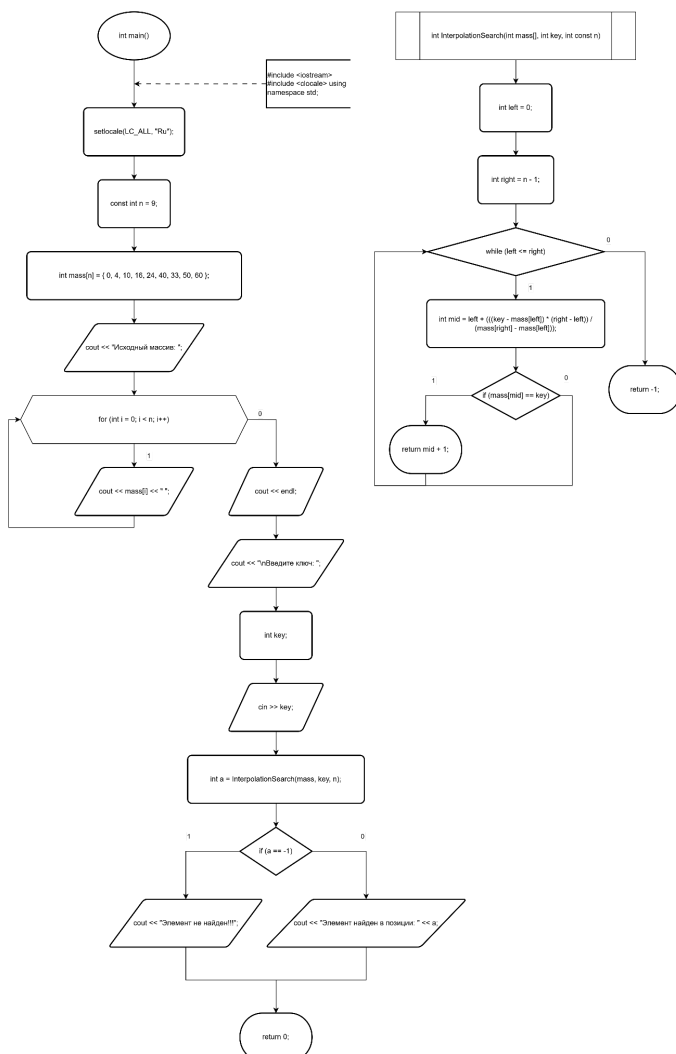
$y_0 = \text{arr}[\text{low}]$, $y_1 = \text{arr}[\text{high}]$ — значения на концах диапазона,

$x_0 = \text{low}$, $x_1 = \text{high}$ — соответствующие индексы,

i — предполагаемый индекс (позиция) искомого значения.

2. Выполняется сравнение с ключом для проверки наличия искомого элемента в найденном месте.
3. Если элемент найден, то поиск завершается, иначе проверяется, в какой половине массива может находиться искомый элемент, и процесс повторяется для соответствующей половины.

3. Блок-схема



4. Код

```
1. #include <iostream>
2. #include <locale>
3. using namespace std;
4.
5. int InterpolationSearch(int mass[], int key, int const n)
6. {
7.     int left = 0;
8.     int right = n - 1;
9.
10.    while (left <= right)
11.    {
12.        int mid = left + (((key - mass[left]) * (right - left)) / (mass[right] - mass[left]));
13.
14.        if (mass[mid] == key)
15.        {
16.            return mid + 1;
17.        }
18.        else if (mass[mid] < key)
19.        {
20.            left = mid + 1;
21.        }
22.        else
23.        {
24.            right = mid - 1;
25.        }
26.    }
27.    return -1;
28. }
29.
30. int main()
31. {
32.     setlocale(LC_ALL, "Ru");
33.     const int n = 9;
34.     int mass[n] = { 0, 4, 10, 16, 24, 40, 33, 50, 60 };
35.
36.     cout << "Исходный массив: ";
37.     for (int i = 0; i < n; i++)
38.     {
39.         cout << mass[i] << " ";
40.     }
41.     cout << endl;
42.
43.     cout << "\nВведите ключ: ";
44.     int key;
45.     cin >> key;
46.
47.     int a = InterpolationSearch(mass, key, n);
48.
49.     if (a == -1)
50.     {
51.         cout << "Элемент не найден!!!";
52.     }
53.     else
```



```

54. {
55.     cout << "Элемент найден в позиции: " << a;
56. }
57.
58. return 0;
59. }

```

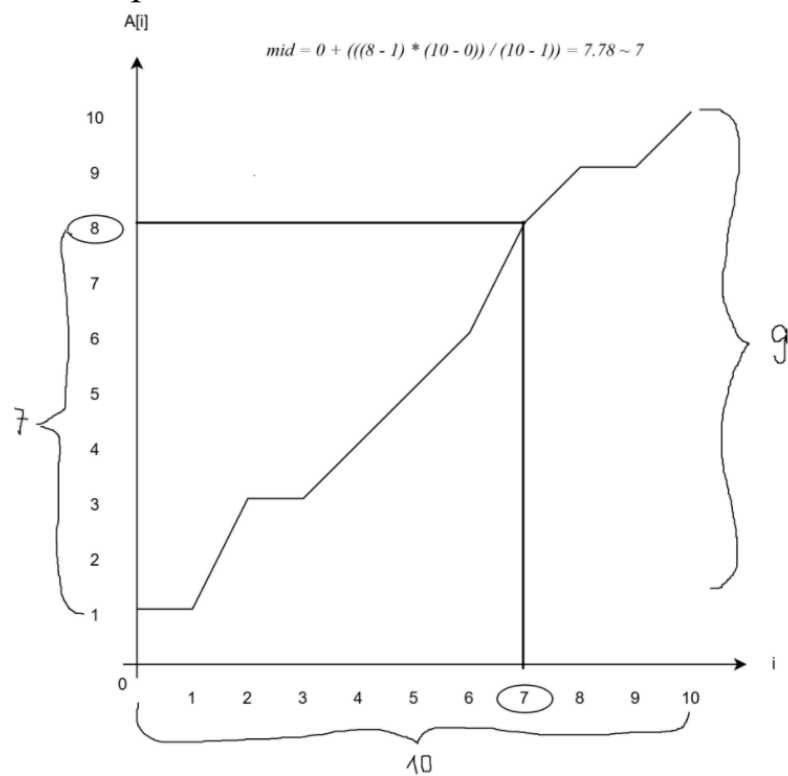
5. Результат выполнения программы

```

Консоль отладки Microsoft Vi × + ▾
Исходный массив: 0 4 10 16 24 40 33 50 60
Введите ключ: 10
Элемент найден в позиции: 3
C:\Users\elenalapteva\source\repos\test_for_
пом. 0 (0x0)

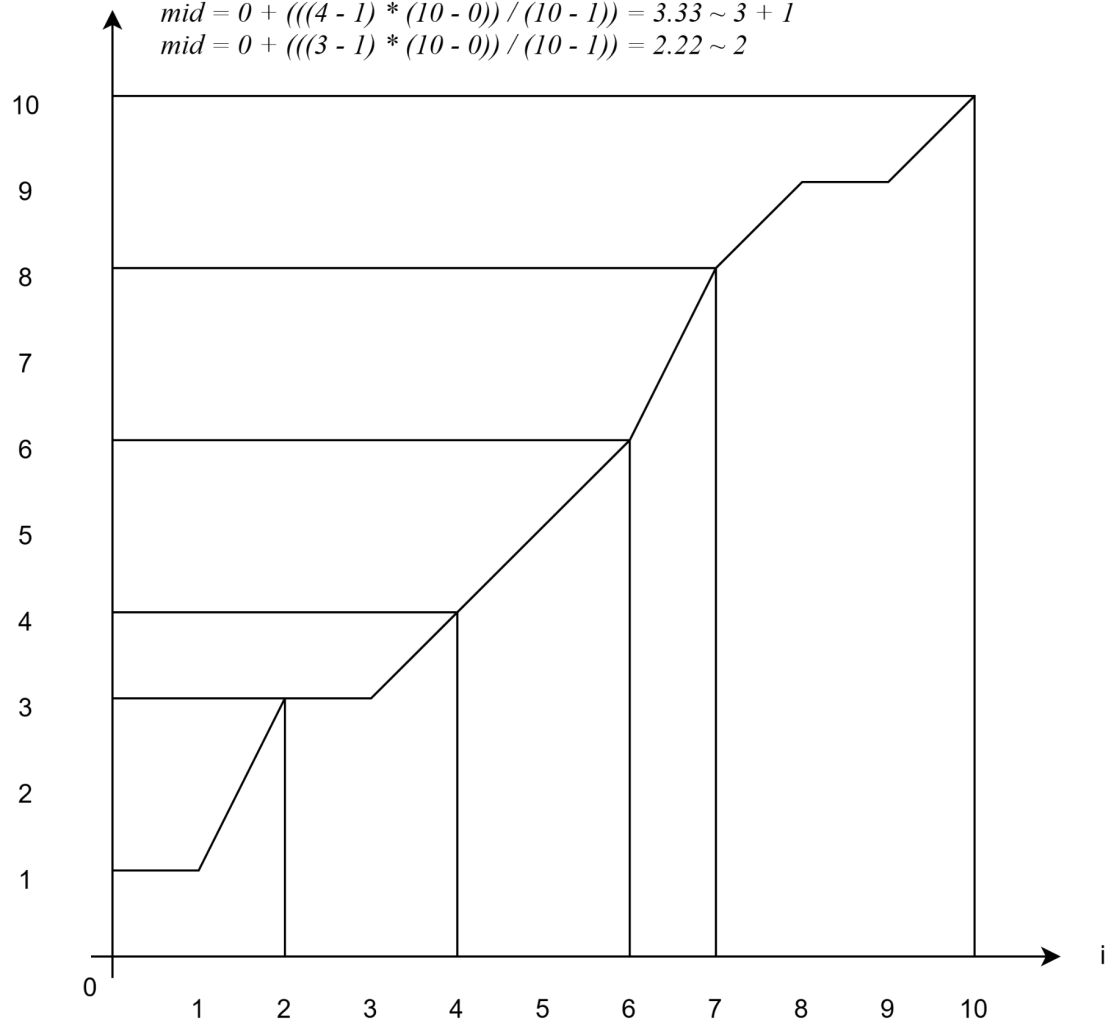
```

6. Графическое решение



+ дз найти 5 эл вычислениями

$$\begin{aligned} mid &= 0 + (((8 - 1) * (10 - 0)) / (10 - 1)) = 7.78 \sim 7 \\ mid &= 0 + (((6 - 1) * (10 - 0)) / (10 - 1)) = 5.55 \sim 5 + 1 \\ mid &= 0 + (((10 - 1) * (10 - 0)) / (10 - 1)) = 10 \\ mid &= 0 + (((4 - 1) * (10 - 0)) / (10 - 1)) = 3.33 \sim 3 + 1 \\ mid &= 0 + (((3 - 1) * (10 - 0)) / (10 - 1)) = 2.22 \sim 2 \end{aligned}$$



IV. Прямой поиск подстроки в строке

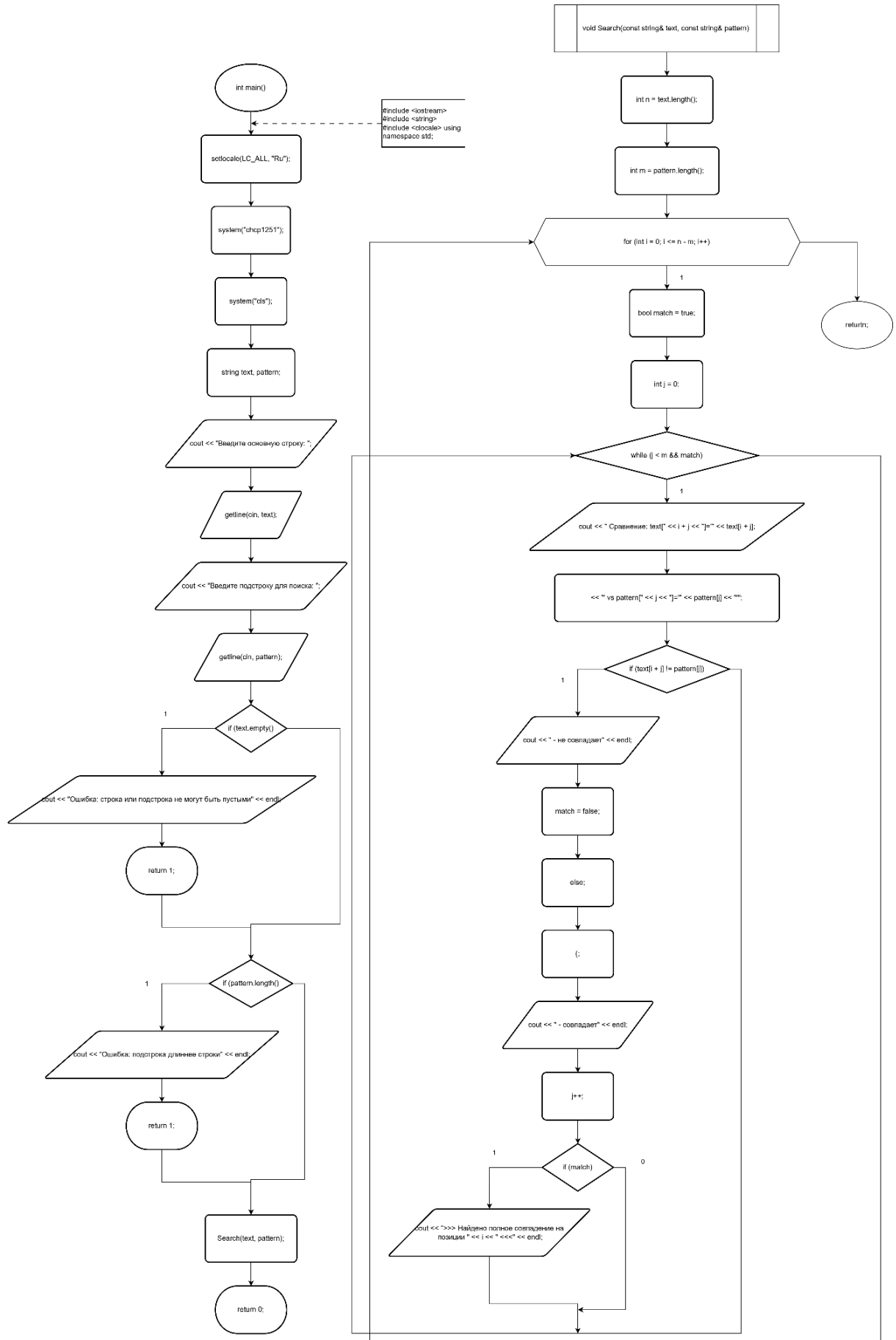
1. Постановка задачи

Найти подстроку в строке.

2. Анализ задачи

Основная идея алгоритма прямым поиском заключается в посимвольном сравнении строки с подстрокой. В начальный момент происходит сравнение первого символа строки с первым символом подстроки, второго символа строки со вторым символом подстроки и т. д.

3. Блок-схема



4. Код

```
1. #include <iostream>
2. #include <string>
3. #include <locale>
4. using namespace std;
5.
6. void Search(const string& text, const string& pattern)
7. {
8.     int n = text.length();
9.     int m = pattern.length();
10.
11.    for (int i = 0; i <= n - m; i++)
12.    {
13.        bool match = true;
14.        int j = 0;
15.
16.        while (j < m && match)
17.        {
18.            cout << " Сравнение: text[" << i + j << "]=" << text[i + j]
19.                << " vs pattern[" << j << "]=" << pattern[j] << " ";
20.
21.            if (text[i + j] != pattern[j])
22.            {
23.                cout << " - не совпадает" << endl;
24.                match = false;
25.            }
26.            else
27.            {
28.                cout << " - совпадает" << endl;
29.                j++;
30.            }
31.        }
32.
33.        if (match)
34.        {
35.            cout << ">>> Найдено полное совпадение на позиции " << i << " <<<<" << endl;
36.        }
37.    }
38. }
39.
40. int main()
41. {
42.     setlocale(LC_ALL, "Ru");
43.     system("chcp1251");
44.     system("cls");
45.
46.     string text, pattern;
47.
48.     cout << "Введите основную строку: ";
49.     getline(cin, text);
50.
51.     cout << "Введите подстроку для поиска: ";
52.     getline(cin, pattern);
53.
```

```
54.  if (text.empty() || pattern.empty())
55.  {
56.      cout << "Ошибка: строка или подстрока не могут быть пустыми" << endl;
57.      return 1;
58.  }
59.
60.  if (pattern.length() > text.length())
61.  {
62.      cout << "Ошибка: подстрока длиннее строки" << endl;
63.      return 1;
64.  }
65.
66.  Search(text, pattern);
67.  return 0;
68. }
69.
```

5. Результат выполнения программы

```
Введите основную строку: void Search(const string& text, const string& pattern)
Введите подстроку для поиска: const
Сравнение: text[0]='v' vs pattern[0]='c' - не совпадает
Сравнение: text[1]='o' vs pattern[0]='c' - не совпадает
Сравнение: text[2]='i' vs pattern[0]='c' - не совпадает
Сравнение: text[3]='d' vs pattern[0]='c' - не совпадает
Сравнение: text[4]=' ' vs pattern[0]='c' - не совпадает
Сравнение: text[5]='S' vs pattern[0]='c' - не совпадает
Сравнение: text[6]='e' vs pattern[0]='c' - не совпадает
Сравнение: text[7]='a' vs pattern[0]='c' - не совпадает
Сравнение: text[8]='r' vs pattern[0]='c' - не совпадает
Сравнение: text[9]='c' vs pattern[0]='c' - совпадает
Сравнение: text[10]='h' vs pattern[1]='o' - не совпадает
Сравнение: text[10]='h' vs pattern[0]='c' - не совпадает
Сравнение: text[11]='(' vs pattern[0]='c' - не совпадает
Сравнение: text[12]='c' vs pattern[0]='c' - совпадает
Сравнение: text[13]='o' vs pattern[1]='o' - совпадает
Сравнение: text[14]='n' vs pattern[2]='n' - совпадает
Сравнение: text[15]='s' vs pattern[3]='s' - совпадает
Сравнение: text[16]='t' vs pattern[4]='t' - совпадает
>>> Найдено полное совпадение на позиции 12 <<<
Сравнение: text[13]='o' vs pattern[0]='c' - не совпадает
Сравнение: text[14]='n' vs pattern[0]='c' - не совпадает
Сравнение: text[15]='s' vs pattern[0]='c' - не совпадает
Сравнение: text[16]='t' vs pattern[0]='c' - не совпадает
Сравнение: text[17]=' ' vs pattern[0]='c' - не совпадает
Сравнение: text[18]='s' vs pattern[0]='c' - не совпадает
Сравнение: text[19]='t' vs pattern[0]='c' - не совпадает
Сравнение: text[20]='r' vs pattern[0]='c' - не совпадает
Сравнение: text[21]='i' vs pattern[0]='c' - не совпадает
Сравнение: text[22]='n' vs pattern[0]='c' - не совпадает
Сравнение: text[23]='g' vs pattern[0]='c' - не совпадает
Сравнение: text[24]='&' vs pattern[0]='c' - не совпадает
Сравнение: text[25]=' ' vs pattern[0]='c' - не совпадает
Сравнение: text[26]='t' vs pattern[0]='c' - не совпадает
Сравнение: text[27]='e' vs pattern[0]='c' - не совпадает
Сравнение: text[28]='x' vs pattern[0]='c' - не совпадает
Сравнение: text[29]='t' vs pattern[0]='c' - не совпадает
Сравнение: text[30]=' ,' vs pattern[0]='c' - не совпадает
Сравнение: text[31]=' ' vs pattern[0]='c' - не совпадает
Сравнение: text[32]='c' vs pattern[0]='c' - совпадает
Сравнение: text[33]='o' vs pattern[1]='o' - совпадает
Сравнение: text[34]='n' vs pattern[2]='n' - совпадает
Сравнение: text[35]='s' vs pattern[3]='s' - совпадает
Сравнение: text[36]='t' vs pattern[4]='t' - совпадает
>>> Найдено полное совпадение на позиции 32 <<<
Сравнение: text[33]='o' vs pattern[0]='c' - не совпадает
Сравнение: text[34]='n' vs pattern[0]='c' - не совпадает
Сравнение: text[35]='s' vs pattern[0]='c' - не совпадает
Сравнение: text[36]='t' vs pattern[0]='c' - не совпадает
Сравнение: text[37]=' ' vs pattern[0]='c' - не совпадает
Сравнение: text[38]='s' vs pattern[0]='c' - не совпадает
Сравнение: text[39]='t' vs pattern[0]='c' - не совпадает
Сравнение: text[40]='r' vs pattern[0]='c' - не совпадает
Сравнение: text[41]='i' vs pattern[0]='c' - не совпадает
Сравнение: text[42]='n' vs pattern[0]='c' - не совпадает
Сравнение: text[43]='g' vs pattern[0]='c' - не совпадает
Сравнение: text[44]='&' vs pattern[0]='c' - не совпадает
Сравнение: text[45]=' ' vs pattern[0]='c' - не совпадает
Сравнение: text[46]='p' vs pattern[0]='c' - не совпадает
Сравнение: text[47]='a' vs pattern[0]='c' - не совпадает
Сравнение: text[48]='t' vs pattern[0]='c' - не совпадает
Сравнение: text[49]='t' vs pattern[0]='c' - не совпадает
```